VIVA DEFENSE GUIDE | PRESENTATION CHEAT-SHEET | DCDS ESE PROJECT (2025–2026) | 99/99 PYTEST PASSED

QueryMind 2.0: Academic Presentation & Viva Defense Study Guide

Target Course: Data Communication & Database Systems (DCDS) **Branch:** `querymind-2.0` | **Date:** October 2026

1. The 2-Minute Elevator Pitch (How to Introduce Your Project)

MEMORIZE AND DELIVER THIS OPENING STATEMENT (60–90 SECONDS)

"Good morning, esteemed examiners. Our project is **QueryMind 2.0**, a voice-enabled relational database query system. While voice provides an accessible, hands-free input modality, our core engineering focus is strictly on **Relational Database Management Systems (RDBMS)**.

Non-technical stakeholders—like educators and academic administrators—struggle to write SQL and navigate complex schemas. Meanwhile, commercial LLM wrappers suffer from stochastic hallucinations, unpredictable table selection, and destructive query risks.

QueryMind 2.0 solves this by introducing a **deterministic, multi-tier pipeline**:

1. We capture speech using the browser-native W3C Web Speech API with **strict pre-execution review**—audio stop never auto-executes queries.
2. We pass speech through an explicit **5-state Intent Classification Engine** that halts conversational greetings and out-of-scope talk before SQL synthesis.
3. We validate queries mathematically using **SQLGlot Abstract Syntax Trees (AST)** to enforce read-only SELECT rules and inject defensive `LIMIT 100` row ceilings.
4. We execute queries across a **Dual-Engine Database Adapter** supporting Cloud Supabase PostgreSQL 15+ with Row-Level Security (RLS) and local SQLite in Write-Ahead Logging (WAL) mode.
5. We expose database internals through **live Mermaid ER diagrams** and **EXPLAIN QUERY PLAN execution profiling**.

Our system is verified through **99 automated pytest tests with a 100% pass rate**. Let us demonstrate how it works."

2. Slide-by-Slide Presentation Script (What to Say on Each Slide)

Slide 1: Title Slide (QueryMind 2.0: Voice-Enabled Database Query System)

What to Say: "Respected examiners, this is QueryMind 2.0. Our course is DCDS. Our project integrates voice natural language processing with rigorous relational database principles, AST security governance, and cloud Supabase PostgreSQL."

Slide 2: Problem Statement & Motivation

What to Say: "Why did we build this? Two reasons: First, writing SQL joins and aggregations is too difficult for ordinary users. Second, existing AI wrappers are dangerous—they hallucinate non-existent tables or emit DROP TABLE statements. Even worse, our baseline had a critical defect: speaking *'hello how are you'* generated an arbitrary query `SELECT * FROM exam_scores LIMIT 50` with a fake 'Safe' badge. We set out to fix this permanently."

Slide 3: Objectives & Project Scope

What to Say: "Our objectives were five-fold: zero-install browser voice capture, deterministic 5-state intent gating, dual-engine RDBMS support (PostgreSQL + SQLite), mathematical SQLGlot AST validation, and educational DBMS transparency via live Mermaid ER diagrams and EXPLAIN query plan profiling."

Slide 4: System Overview / Proposed Solution

What to Say: "Here is our decoupled architecture. Notice the critical principle: **Microphone stop NEVER automatically executes queries**. The voice stream is normalized, classified for intent, translated to SQL, parsed into an AST, and displayed in an interactive editor. The user must manually click [Run Query]. We also eliminated misleading single 'Safe' badges by providing separate badges for Speech Recognized, Intent Validated, and AST Validated."

Slide 5: Requirements & Technology Stack

What to Say: "We use Python 3.11 with Flask for the REST backend, Vanilla JavaScript with the Web Speech API on the frontend without heavy framework overhead, SQLite in WAL mode for local zero-config CI, Supabase PostgreSQL 15+ for enterprise cloud deployment, SQLGlot for AST parsing, and Pytest running our 99 tests."

Slide 6: Database Design — ER Diagram

What to Say: "Our relational database models academic records: `students`, `subjects`, `marks`, and `attendance`. Foreign keys enforce referential integrity with cascading deletes. In our live system, the ER diagram is generated dynamically using Mermaid.js by querying SQLite catalog pragmas and PostgreSQL constraints."

Slide 7: Relational Schema & Normalization (1NF, 2NF, 3NF)

What to Say: "Our schema strictly satisfies Third Normal Form (3NF). 1NF is satisfied because all attributes are atomic scalar values with designated primary keys. 2NF is satisfied because transactional records (`marks` and `attendance`) use composite candidate keys `(student_id, subject_id)`, and subject attributes like credits are decoupled to `subjects`, eliminating partial dependencies. 3NF is satisfied because student demographics reside strictly in `students`, eliminating transitive dependencies like mark_id → student_id → division."

Slide 8: Implementation & SQL Operations

What to Say: "Our SQL generator translates voice queries into multi-table relational joins, aggregate grouping, and threshold filtering (like attendance below 75%). Importantly, our AST parser auto-injects `LIMIT 100` ceilings when queries omit limits, protecting database memory buffer pools from exhaustion."

Slide 9: Results / Demonstration

What to Say: "In our live workspace, speaking a greeting like *'hello how are you'* returns an out-of-scope status, clears the SQL editor, hides the AST badge, and suggests valid questions. When a valid query runs, our interactive [Explain Plan] drawer reveals whether the database used a full table scan or an index lookup."

Slide 10: Testing & Validation (99/99 Tests Passed)

What to Say: "We adhere to a strict test-driven development loop. Our automated test suite grew from 59 to **99 passing tests** across 5 test files in 3.99 seconds, including 40 regression tests in `test_intent_regression.py` verifying our greeting bug fix."

Slide 11: Conclusion & Future Scope

What to Say: "We have proven that natural language voice interaction with databases can be achieved safely without LLM hallucinations. For future scope, we plan to add offline Vosk WebAssembly models for air-gapped institutions and Redis multi-turn session memory."

Slide 12: References & Q&A

What to Say: "We cited primary sources including E. F. Codd's 1970 relational model paper, W3C Web Speech specifications, and PostgreSQL 15 documentation. We are now eager to answer your questions and demonstrate the live system."

3. Core DBMS Concepts Tested in Viva (With Exact Answers)

Q1: How do you prove your database is in 3NF? What are the functional dependencies?

Model Answer: "We established the functional dependency set F:

- `student_id → { name, division, email }`
- `subject_id → { subject_name, credits }`
- `{ student_id, subject_id } → { marks, attendance_percentage }`

1NF: All attributes contain atomic scalar values with unique primary keys.
2NF: In `marks` and `attendance`, the candidate key is composite `(student_id, subject_id)`. All non-key attributes (marks, percentage) depend on the *entire* key. Subject details (credits) were factored out into `subjects`, avoiding partial functional dependencies.
3NF: There are no transitive dependencies ($X \to Y \land Y \to Z$). Student division depends solely on `student_id`; storing division in `marks` would create a transitive dependency ($mark_id \to student_id \to division$). Because we only store foreign keys, 3NF holds strictly."

Q2: Why did you use SQLGlot for AST validation instead of regular expressions?

Model Answer: "Regular expressions rely on lexical string pattern matching. Attackers easily bypass regex using SQL comment obfuscation (`SEL/**/ECT`), whitespace variations, or semicolon chaining. SQLGlot tokenizes the SQL string into an in-memory Abstract Syntax Tree. We inspect the root node: if it is not an instance of `sqlglot.expressions.Select`, or if there is more than one root expression, the query is rejected mathematically before it ever reaches the database driver."

Q3: What is the difference between SQLite WAL mode and traditional rollback journal?

Model Answer: "In traditional rollback journal mode, writing locks the entire database file, preventing concurrent reads. In Write-Ahead Logging (`PRAGMA journal_mode = WAL`), changes are written sequentially to a separate `-wal` file. Readers read from the main database and WAL simultaneously without acquiring write locks. This enables high-concurrency multi-reader access while maintaining full ACID durability."

Q4: How does EXPLAIN QUERY PLAN work in your system?

Model Answer: "When a user executes or inspects a query, we run `EXPLAIN QUERY PLAN <sql>`. SQLite's query optimizer returns an execution tree. We parse the output to detect:

- `FULL_TABLE_SCAN`: Scanning every disk page sequentially (e.g. `SCAN attendance`).
- `INDEX_SEARCH`: Using a B-Tree index seek in $\mathcal{O}(\log N)$ time (e.g. `SEARCH students USING INTEGER PRIMARY KEY`).
- `TEMP_BTREE_SORT`: Constructing an in-memory B-Tree to satisfy `ORDER BY` or `GROUP BY`.

We display this in an interactive drawer so users can see how indexing avoids costly sequential scans."

Q5: How does Supabase Row-Level Security (RLS) work?

Model Answer: "RLS moves authorization from application code into the PostgreSQL kernel. When a student queries `SELECT * FROM marks`, the policy checks `USING (student_id IN (SELECT s.student_id FROM students s JOIN users u ON s.user_id = u.user_id WHERE u.auth_uid = auth.uid()))`. Even if a student crafts a malicious query trying to see other students' marks, PostgreSQL returns an empty set or only their own records at the engine level."

4. The Critical Bug Defense: The Greeting Bug Autopsy

THE QUESTION EXAMINERS LOVE: "WHAT WAS THE HARDEST BUG YOU SOLVED?"

Answer with Confidence:

"Our most critical bug was in conversational speech handling. When a user spoke greetings like *'hello how are you'* or *'tell me a joke'*, the interface generated:

```
SELECT * FROM "exam_scores" LIMIT 50
```

and displayed two green badges: *'SQL generated successfully'* and *'AST Validated • Read-Only SELECT • Safe'*.

Why it happened (Root Cause Autopsy):

1. In `nlp/sql_generator.py` line 402 inside `generate_sql_for_dataset()`, there was an arbitrary fallback: whenever no keywords matched, it returned `SELECT * FROM "{table_name}" LIMIT 50` with `matched: True`.
2. The API gateway in `app.py` routed all transcriptions straight to SQL synthesis without any prior intent check.
3. In `static/js/voice_sql.js`, when an input failed to match, the `#sqlEditor` textarea was never cleared, leaving previous SQL executable.

How we solved it:

1. Removed the arbitrary fallback in `nlp/sql_generator.py` so unmatched queries strictly return `matched: False, sql: None`.
2. Created an explicit 5-state Intent Classifier in `nlp/intent_classifier.py` (`OUT_OF_SCOPE`, `AMBIGUOUS_QUERY`, `UNSUPPORTED_OPERATION`, `EMPTY_INPUT`, `DATABASE_QUERY`). Greetings return `status: "out_of_scope"` with guidance suggestions.
3. Updated `static/js/voice_sql.js` to clear `#sqlEditor`, hide AST badges, and render interactive suggestion chips.
4. Wrote 40 regression tests in `tests/test_intent_regression.py` proving that conversational inputs generate zero SQL and cannot execute queries."

5. Top 25 Viva Questions & Rapid-Fire Model Answers

Examiner Question	Bullet-Proof Technical Model Answer
1. What happens if a user says "Drop table students"?	The Intent Classifier classifies it as `UNSUPPORTED_OPERATION`. Even if someone typed it into the editor, SQLGlot AST validation blocks it because `exp.Drop` is not in our `exp.Select` whitelist.
2. Does the microphone auto-execute queries when you stop speaking?	**No, never.** Audio stop only populates the editable textarea. The user must review the recognized speech, inspect the generated SQL, and click `[Run Query]` manually.
3. Why do you have two databases (SQLite and PostgreSQL)?	We built a unified `DatabaseAdapter`. SQLite (in WAL mode) provides zero-config local development and CI testing. Supabase PostgreSQL provides cloud scalability, connection pooling, and Row-Level Security.
4. What prevents SQL injection via comments like `SELECT 1; --` ?	SQLGlot parses query tokens into AST nodes and strips out comment tokens. Semicolons followed by second statements produce two AST root nodes, violating our single-statement invariant.
5. How does the system handle numbers spoken as words?	Our text normalizer in `nlp/sql_generator.py` uses regex token replacement to convert *"seventy five"* to `75`, *"five"* to `5`, and *"percent"* to `%` before slot filling.
6. Why do you inject LIMIT 100 into queries?	To protect database buffer pools and memory from denial-of-service query exhaustion if a user queries a table with millions of rows without a LIMIT clause.
7. What is an ambiguous query example?	*"Show me the data"* or *"Find records"*. It classifies as `AMBIGUOUS_QUERY` and prompts: *"Which table or entity would you like to query? (students, marks, attendance)"*.
8. How is the Mermaid ER diagram generated?	In `db/database.py`, we introspect SQLite schema pragmas (`PRAGMA table_info` and `PRAGMA foreign_key_list`), loop through tables and relationships, and dynamically format valid Mermaid.js syntax.
9. What is the role of `ON DELETE CASCADE` in your schema?	When a student record is deleted from `students`, all related child records in `marks` and `attendance` are automatically deleted, maintaining referential integrity without orphaned rows.
10. How do you handle password storage?	Passwords in `app_users` are hashed using SHA-256 (`hashlib.sha256`) with salts. Password hashes are masked and blocked from student-level query responses.
11. How many automated tests do you have?	We have **99 automated pytest tests** across 5 test files, running in 3.99 seconds with a 100% pass rate.
12. What is a generated column in PostgreSQL?	In our Supabase schema, attendance percentage is a stored generated column: `percentage GENERATED ALWAYS AS (ROUND((attended::numeric / total_classes::numeric) * 100, 2)) STORED`.
13. How does the system handle missing microphone permissions?	The Web Speech API triggers `onerror` with `event.error === 'not-allowed'`. We display a user-friendly alert asking them to allow mic access, without crashing the app.
14. What are the three node types in your execution plan analyzer?	`FULL_TABLE_SCAN` (scanning every row), `INDEX_SEARCH` (B-Tree lookup), and `TEMP_BTREE_SORT` (sorting in temporary memory).
15. Can QueryMind 2.0 work on custom uploaded CSV datasets?	Yes. Administrators can upload CSV files in the Admin Dashboard, which automatically creates SQLite tables, registers them in `dataset_registry`, and makes them queryable via NLP.

6. Live Demonstration Cheat-Sheet (Step-by-Step Demo Flow)

> **THE EXACT 5-STEP DEMO TO SHOW THE EVALUATORS**
> Follow this sequence to showcase all system capabilities within 3 minutes:

1. **Demo 1: The Critical Bug Resolution (Show Intent Gating)**
   • Click the microphone icon and speak clearly: *"Hello how are you"*.
   • **What to point out:** Notice that **zero SQL** was generated. The editor is clean, the AST badge is suppressed, and the system displays `status: "out_of_scope"` with suggestion chips (*"Show all students"*, *"Find students with attendance below 75%"*). Explain that QueryMind 2.0 no longer generates arbitrary queries.
2. **Demo 2: Ambiguous Query Clarification**
   • Speak: *"Show me the data"*.
   • **What to point out:** The system classifies this as `AMBIGUOUS_QUERY` and politely asks: *"Your request is too ambiguous. Which table or entity would you like to query?"*
3. **Demo 3: Valid Voice Relational Join & Filter**
   • Speak: *"Students with attendance below 75 percent"*.
   • **What to point out:** Show how spoken *"seventy five percent"* normalized to `75%`. Show the synthesized SQL featuring `JOIN attendance` and `WHERE a.attendance_percentage < 75`. Show the **AST Validated** badge. Then click **[Run Query]** to display the formatted table.
4. **Demo 4: Internal DBMS Profiler (EXPLAIN Plan)**
   • Click the blue **[Explain Plan]** drawer button on the results panel.
   • **What to point out:** Show the execution plan tree. Point out `SCAN a` (Full table scan on attendance) and `SEARCH s USING INTEGER PRIMARY KEY` (B-Tree index lookup on students). Explain the time complexity differences to the examiner.
5. **Demo 5: Live Database Schema Explorer & Dynamic Mermaid ER**
   • Navigate to `/database-explorer`.
   • **What to point out:** Show the live Mermaid.js ER diagram rendered dynamically from SQLite catalog pragmas with 1:N cardinality connectors.

7. Final Presentation Checklist

- [x] **Flask Server Running:** `python app.py` running at `http://127.0.0.1:5000`.
- [x] **Browser Ready:** Chrome or Edge open at `http://127.0.0.1:5000/voice-sql`.
- [x] **Microphone Tested:** Verify browser microphone permission is allowed.
- [x] **Slides Ready:** `QueryMind_2_DCDS_ESE_Presentation.pptx` open in PowerPoint.
- [x] **Report Available:** 30-page PDF report `QueryMind_2_Final_Implementation_Report.pdf` open in Acrobat.
- [x] **Test Results Verified:** 99/99 pytest tests passing in 3.99 seconds.